

INFORME DE AUDITORÍA

BDGB v3 — Binary Dynamics Grid Brain

Rejilla binaria 16×16 con dinámica Kaprekar, cifrado de flujo BDGB-Cipher v1, motor semántico, NLP, aprendizaje, agentes y puente Python.

Repositorio	https://github.com/1974juan0612-rgb/bdgb
Lenguaje	C99 + Python 3
Líneas C	~2,500 (núcleo + crypt)
Líneas Python	~375 (bridge + scraper)
Nodos	256 (16×16 grid, 8-bit)
Versión auditada	v3 — Junio 2026
Tests	19/19 PASS

1. Descripción General del Proyecto

BDGB (Binary Dynamics Grid Brain) v3 es un sistema de conocimiento geométrico basado en una rejilla binaria de 16×16 nodos (256 nodos, 8-bit cada uno). Cada nodo representa un valor 0–255 cuyas propiedades se derivan de su representación binaria: densidad (popcount), simetría, tipo geométrico (esquina/borde/interior), radio, clase dinámica y atractor asociado.

La dinámica Kaprekar binaria asigna a cada nodo un sucesor (restando el orden ascendente del descendente de sus bits), generando cuencas de atracción que convergen en puntos fijos. Este modelo matemático subyace tanto al motor de búsqueda semántica como al nuevo sistema de cifrado.

Componentes del sistema:

Componente	Descripción	Estado
Núcleo geométrico	Grid 16×16, nodos de 4 bytes, propiedades O(1)	Funcional (100%)
Regla dinámica	Sucesor Kaprekar binario, detección de atractores	Funcional (100%)
Semántica	Hash index (256 buckets) para relación nodo→concepto	Funcional (90%)
Grafo de conceptos	Hash index para aristas entre conceptos	Funcional (85%)
Motor de búsqueda	Híbrido: semántico + propiedades + atractor	Funcional (80%)
NLP	20 términos fijos, tokenizador, consultas híbridas	Funcional (60%)
Aprendizaje	Refuerzo y decaimiento con persistencia a disco	Funcional (70%)
Agentes	Registry JSON, pipeline declarativo, supervisor tick	Estructural (50%)
BDGB-Cipher v1	Cifrado de flujo sin dependencias externas	NUEVO (100%)
Python bridge	CLI wrapper para integración desde Python	Funcional (90%)
Scraper	Mock + pytrends para inyección de tendencias	Funcional (70%)

2. Sistema de Cifrado — BDGB-Cipher v1

BDGB-Cipher v1 es un cifrado de flujo (stream cipher) diseñado completamente sobre el modelo matemático BDGB, sin utilizar ninguna biblioteca criptográfica externa. Implementado en 199 líneas de C99 (archivo `crypt/bdgb_crypt.c`).

2.1 Estado interno

El estado del cifrado son 4 palabras de 32 bits (`uint32_t wseed[4]`), un vector de inicialización de 8 bytes y un contador de 64 bits. Total: 32 bytes de estado.

2.2 S-box geométrica

La función `bdgb_sbox()` no es una tabla lookup: computa en tiempo real las propiedades geométricas del nodo en la rejilla 16×16 (densidad, radio, simetría, tipo_geom), las combina con los pasos al atractor y el popcount, y produce un byte no lineal. Esto hace que la S-box dependa íntegramente del modelo BDGB subyacente.

2.3 Derivación de clave

La función `derive_seed_32()` procesa cada byte de la contraseña + IV mediante una mezcla aritmética (XOR, suma, rotación) aplicando la función `mix32()` en 8 rondas. Cada ronda mezcla progresivamente las 4 palabras de estado con retroalimentación cruzada.

2.4 Generación de keystream

Por cada byte de keystream, las 4 palabras de estado se mezclan con retroalimentación del contador (counter de 64 bits). Cada byte de salida depende de los 4 words del estado actual y del contador. Tras cada byte, el contador se incrementa, garantizando divergencia.

2.5 Formato de archivo .bdgb

Offset	Tamaño	Campo	Descripción
0–3	4 B	Magic	"BDGB" (firma)
4	1 B	Versión	0x01
5–12	8 B	IV	Vector de inicialización aleatorio

Offset	Tamaño	Campo	Descripción
13+	variable	Ciphertext	Plaintext XOR keystream

2.6 Seguridad y dependencias

El cifrado tiene **cero dependencias externas**: no usa OpenSSL, no usa nada del sistema operativo. Es C99 puro. La seguridad se basa en la no linealidad de la S-box geométrica y en la mezcla aritmética de las 4 palabras de estado. El espacio de claves es de 2^{128} (4×32 bits). El desafío público (crypt/challenge/) invita a la comunidad a intentar romperlo, validando así su robustez.

3. Desafío de Desenscriptado Comunitario

Se ha publicado un desafío público en crypt/challenge/. El archivo secret.bdgb (445 bytes) contiene un mensaje cifrado con BDGB-Cipher v1 usando una contraseña alfanumérica (a-z, 0-9).

Archivos del desafío:

Archivo	Descripción
crypt/challenge/secret.bdgb	Archivo cifrado de 445 bytes
crypt/challenge/README_CHALLENGE.md	Reglas, pistas y documentación del desafío
crypt/challenge/CHALLENGE_BUNDLE.md	Paquete compacto para análisis con IA local

El desafío permite cualquier método: fuerza bruta, criptoanálisis, ingeniería inversa. El objetivo es extraer el mensaje original y demostrar la contraseña utilizada. Si se encuentra una vulnerabilidad en el diseño, se otorgará crédito en el repositorio y mención en la documentación oficial.

4. Suite de Tests

La batería de tests (tests/test_bdgb.c) cubre todos los módulos del sistema. Resultado: **19/19 PASS, 0 FAIL**.

Tests ejecutados:

#	Test	Módulo	Resultado
1	grid_size	Constantes del grid (8 bits, 16×16, 256 nodos)	PASS
2	node_creation	Creación de 256 nodos con coordenadas correctas	PASS
3	popcount	Conteo de unos (0, 1, 3, 0xFF, 0x0F)	PASS
4	symmetry	Detección de simetría (0, 0xFF, 0x81, 0x1B)	PASS
5	geom_types	Tipos geométricos en grid 16×16	PASS
6	dynamic_rule	Regla Kaprekar binaria (punto fijo, mapeo)	PASS
7	attractor_search	256 nodos convergen a atractor	PASS
8	storage_roundtrip	Persistencia y carga de 256 nodos	PASS
9	semantics_hash	Hash index en semántica	PASS
10	concept_graph_hash	Hash index en grafo de conceptos	PASS
11	learning_reinforce	Refuerzo (+10) y decaimiento (x0.5)	PASS
12	nlp_parse	Tokenización y mapeo a conceptos	PASS
13	search_hybrid	Búsqueda híbrida + refuerzo	PASS
14	search_by_predicate	Filtro por propiedades (simétrico + interior)	PASS
15	compute_props	Propiedades válidas para 256 nodos	PASS
16	crypt_init	Inicialización del contexto de cifrado	PASS
17	crypt_keystream_differs	Divergencia con distintas claves/IV	PASS
18	crypt_encrypt_decrypt	Ronda completa cifrado/descifrado	PASS
19	crypt_file_roundtrip	Cifrado/descifrado de archivo completo	PASS

Nota: Todos los tests utilizan datos temporales en tests/test_data/ que se limpian antes y después de la ejecución. No hay dependencia de red ni de bibliotecas externas.

5. Arquitectura del Sistema

5.1 Pila tecnológica

Capa	Tecnología	Función
Lenguaje base	C99 (ISO/IEC 9899:1999)	Núcleo de alto rendimiento
Build system	CMake 3.10 + Ninja	Compilación cruzada y rápida
CLI	main.c con argparse manual	Interfaz de línea de comandos
Python Bridge	bdgb_bridge.py (subprocess)	Integración desde Python
Scraper	scraper_trends.py (pytrends)	Inyección de tendencias
GUI	bdgb_gui.py (Tkinter)	Visualización de grid y búsqueda

5.2 Comandos CLI

Comando	Función
<code>--init</code>	Limpia e inicializa datos (256 nodos + semántica demo)
<code>--search &lt;query&gt;</code>	Búsqueda NLP con salida JSON
<code>--add-concept &lt;n&gt; &lt;c&gt;</code>	Añade concepto, semántica
<code>--export-nodes</code>	Dump JSON de todos los nodos con propiedades
<code>--agent-run &lt;id&gt;</code>	Ejecuta pipeline de agente
<code>--encrypt &lt;in&gt; &lt;out&gt;</code>	Cifra archivo con BDGB-KAPREKAR
<code>--decrypt &lt;in&gt; &lt;out&gt;</code>	Descifra archivo
<code>--hash &lt;text&gt;</code>	Genera hash BDGB de 16 bytes

5.3 Variables de entorno y portabilidad

El sistema soporta BDGB_ROOT como variable de entorno para rutas portátiles. El flag --data-path permite especificar el directorio de datos en tiempo de ejecución. El archivo .gitignore excluye build/, __pycache__/, test_data/ y data/.

5.4 Cero dependencias externas en C

Todo el núcleo C, incluyendo el cifrado BDGB-Cipher, se implementa sin ninguna dependencia externa. No se utiliza OpenSSL, libcrypto, ni ninguna biblioteca de terceros. La S-box, la derivación de clave y la generación de keystream se basan únicamente en aritmética de enteros y las propiedades geométricas de la rejilla BDGB.

6. Estructura del Repositorio

Directorios raíz:

Directorio/Archivo	Descripción
<code>include</code>	Headers C: bdgb.h, semantics.h, concept_graph.h, search.h, learning.h, nlp.h, agent.h
<code>src</code>	Implementaciones C: main.c, node.c, grid.c, storage.c, semantics.c, concept_graph.c, search.c, learning.
<code>crypt</code>	Módulo de cifrado: bdgb_crypt.c, bdgb_crypt.h, challenge/
<code>crypt/challenge/</code>	Desafío público: secret.bdgb, README_CHALLENGE.md, CHALLENGE_BUNDLE.md
<code>tests</code>	Suite de tests: test_bdgb.c, test_data/
<code>interface/</code>	Python: bdgb_bridge.py (CLI wrapper), bdgb_gui.py (Tkinter)
<code>scripts</code>	Python: scraper_trends.py (pytrends + mock)
<code>agents</code>	Configuración de agentes: registry.json, youtube-automator/
<code>data/</code>	Datos persistidos: nodes.dat, semantics.dat, concept_edges.dat, usage_*.dat
<code>build</code>	Artefactos de compilación (.exe, .obj)
<code>CMakeLists.txt</code>	Build de CMake (bdgb + bdgb_test)
<code>.gitignore</code>	Exclusiones: build/, __pycache__/, test_data/, data/

Directorio/Archivo	Descripción
auditoria.md	Informe de auditoría previo (v1)

Formatos de datos en disco:

Archivo	Formato	Tamaño registro
nodes.dat	Binario secuencial: id_bits (1B) + x (1B) + y (1B) + flags (1B)	4 B/nodo
semantics.dat	Binario: node_id (1B) + concept_id (2B) + weight (1B) + rel_type (1B)	5 B/arista
concept_edges.dat	Binario: from (2B) + to (2B) + weight (1B) + rel_type (1B)	6 B/arista
concepts.idx	Hash index (256 buckets) para semántica	256 × 32 B
concept_edges.idx	Hash index (256 buckets) para grafo de conceptos	256 × 32 B
usage_*.dat	Binario: ID (1–2B) + contador (4B)	5–6 B/entrada

7. Historial de Versiones

Commits (más recientes):

Hash	Mensaje
21965a3d66	BDGB-Cipher v1 + challenge de descryptado comunitario
4621078d4f01	BDGB-Grid: ignore para build, pycache, test_data, data
1b025882f5a1	BDGB-Grid: salto de prototipo a herramienta tecnica
2506b444e0fa	BDGB-Grid: grafica Python/Tkinter - grid 4x4, busqueda NLP, panel de agentes
726b1d65e1e0	BDGB-Grid: base de agentes - registry, config, supervisor, agent module
61267c4f010e	BDGB-Cipher: motor de lenguaje natural - 20 terminos, tokenizer, consultas hibridas
7d7c839f10e4	BDGB-Grid: primera version - geometria 4x4, semantica, busqueda, aprendizaje

8. Veredicto

Fortalezas:

- Diseño por capas limpio y extensible. Cada módulo tiene su header con tipos y funciones claras. El cambio de BDGB_GRID_BITS expande toda la geometría automáticamente.
- Cero dependencias externas en C, incluso en el módulo de cifrado.
- 19/19 tests pasando, cubriendo todos los módulos incluyendo el nuevo cifrado.
- Hash indices para búsquedas O(1) en semántica y grafo de conceptos.
- Sistema de cifrado novedoso basado únicamente en el modelo BDGB.
- Puerto Python limpio (bdgb_bridge.py) que evita duplicar la lógica del motor C.

Debilidades y riesgos:

- La búsqueda secuencial en find_nodes_by_concept aún recorre todo el archivo en ciertos casos. Con miles de nodos será lento.
- El NLP tiene solo 20 términos fijos y no aprende palabras nuevas.
- La ejecución real de agentes (pipeline) no conecta herramientas externas (Godot, ffmpeg, YouTube). Es una estructura vacía.
- Paths hardcodeados a Windows en los tests. Linux no funciona sin cambios.
- El cifrado BDGB-Cipher v1 no ha sido auditado por criptógrafos profesionales (de ahí el desafío público).

BDGB v3 ha evolucionado de un prototipo conceptual (v1, 16 nodos, 4x4) a una herramienta técnica con 256 nodos, hash indices, cifrado propio, 19 tests y un desafío público. El diseño por capas sigue siendo sólido. La incorporación del cifrado basado en la geometría BDGB representa un hito significativo. Los próximos pasos deberían enfocarse en: (1) portabilidad Linux, (2) expansión del NLP, (3) conexión real de herramientas con agentes, (4) auditoría criptográfica profesional del BDGB-Cipher.

